

Existing Application Analysis and React Planning

BillBhai : Retail Order Processing and Billing System

FDFED M2026 | React Project Lab 1 | Team 44 | Repository: 44_BillBhai

About the application. BillBhai is a retail billing and store management system. A shop owner registers a business, staff sign in under one of seven roles, cashiers create orders at a billing counter, and managers track inventory, deliveries, returns and reports from an admin console. The frontend is built as a set of separate HTML pages with plain JavaScript and no framework. The backend is a NestJS API that the frontend calls for all of its data.

1. Application Flow

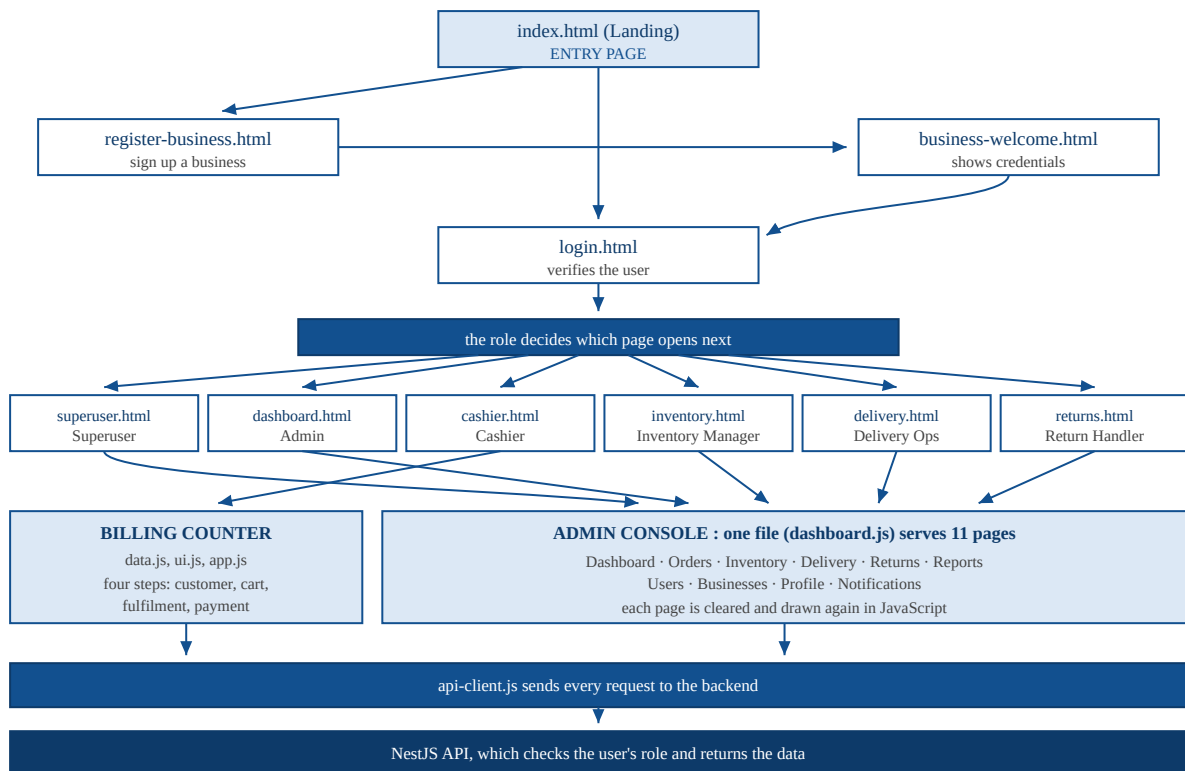


Figure 1 : Page navigation, the redirect by role, the two parts of the application, and the backend.

Main workflows

Workflow	Steps
Register a business	Landing page, registration form, then a welcome page that shows the generated login details, and finally the login page.
Sign in	Login form, the account is verified, the role is saved in the browser, and the user is sent to the page meant for that role.
Create an order	Enter the customer phone number, which fills in the details if the customer already exists. Add products to the cart, apply a promo code, choose takeaway or delivery, then complete payment. The order is then saved to the backend.
Manage orders	Open the Orders page, filter by status and payment, sort the list, then edit or delete an order.
Manage stock	Open the Inventory page, look at the stock alerts, add a product or adjust the quantity of an existing one.
Deliveries and returns	On the Delivery page, assign a partner and move a delivery forward. On the Returns page, approve, reject or refund a request.
Reports	Open the Reports page and pick a date range. The summary tables and charts are recalculated.

Where the data comes from. The user enters data through the login and registration forms, the billing counter, and the dialogs for adding orders, products and users. The user views data through the summary cards, charts, the orders, inventory, delivery, returns and users tables, the report summaries and the cart totals. Products, customers, orders, inventory, deliveries, returns, suppliers, businesses and users all come from the backend. Notifications and report totals do not, because they are worked out in the browser.

2. Repeated UI Analysis

Below are the six elements that repeat most across the application, shown as they actually appear on screen.

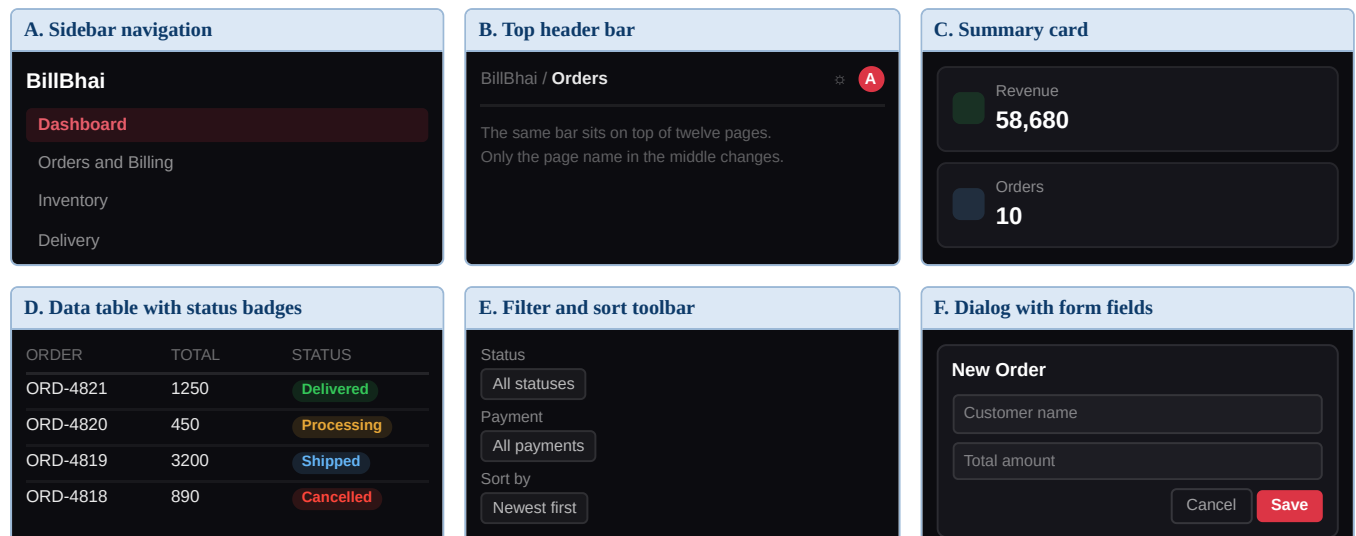


Figure 2 : The six repeated elements as they appear in the running application.

UI Element	Pages / Location	How is it repeated?	Why could reuse be useful?
A. Sidebar navigation	Eleven pages, from Dashboard through to Notifications	The same block of markup is pasted into every file, giving 88 copies of the same menu links	One <code><Sidebar /></code> component. Adding a menu item becomes one edit instead of eleven.
B. Top header bar	The same eleven pages, plus the billing counter page	The menu button, breadcrumb, notification bell and profile dropdown are duplicated twelve times, and only the page name differs	One <code><TopHeader /></code> component where the title is passed in as a property.
C. Summary card	Dashboard, Orders, Delivery and Returns, four on each page	The same icon, label and value layout is written out sixteen times, and only the colour and the numbers change	One <code><StatCard /></code> component covers every version of it.
D. Data table with badges	Dashboard, Orders, Inventory, Delivery, Returns, Reports and Users	The same table is rebuilt separately inside each page's drawing function, and around forty badges are written by hand in the HTML	One <code><DataTable /></code> and one <code><Badge /></code> component. The filtering, sorting, empty message and status colours get written once.
E. Filter and sort toolbar	Orders, Inventory, Delivery, Returns and Reports	The same toolbar is rebuilt in each drawing function, each one wiring up its own change handler	One <code><FilterToolbar /></code> component, with the chosen filter kept as state.
F. Dialog with form fields	Orders, Inventory and Users, plus the login and registration forms	Six separate versions of the dialog, each with its own opening and closing code, and every field paired with an error message by hand	One <code><Modal /></code> and one <code><FormField /></code> component, reused by every form in the product.

React connection: these six elements are effectively the starting list of components for the project. The sidebar, header, card and badge hold no data of their own, which makes them the safest place to begin.

3. Dynamic Behaviour Analysis

Page / Feature	User Action	What Changes?
Landing	Scrolls the page	The top bar gains a shadow and each section fades in as it comes into view.
Login	Submits an empty field	The field shakes and turns red, and an error message appears below the form.
Login	Submits correct details	The button shows a loading spinner, then a tick, then the page changes based on the role.
Register business	Types a password	A password strength meter updates on every keystroke.
Billing counter	Enters a ten digit phone number	The details of an existing customer are filled in automatically and the hint below the field changes.
Billing counter	Submits the customer form	Step one is hidden and step two appears.
Billing counter	Types in the product search box	The whole product list is rebuilt on every keystroke, and a message appears when nothing matches.
Billing counter	Clicks a category button	That button becomes active, the others become inactive, and the product list is filtered.
Billing counter	Clicks a product	The item is added to the cart or its quantity goes up. The cart list, subtotal, discount, total and two buttons all update.
Billing counter	Presses plus or minus in the cart	The quantity changes, the row is removed at zero, and the totals are recalculated.
Billing counter	Applies a promo code	Either an error appears, or the input is replaced by a tag showing the code and the total goes down.
Billing counter	Chooses delivery instead of takeaway	The address and delivery partner fields appear, a delivery charge is added, and the checkout button label changes.
Admin pages	Opens a page while signed in	Menu items the role is not allowed to see are hidden, and buttons the role cannot use are switched off.
Orders	Changes the status, payment or sort control	The table is rebuilt from the filtered and sorted list, or a message says nothing matches.
Orders	Clicks New Order, Edit or Delete	A dialog opens. After saving, the order is sent to the backend and the table and summary cards refresh.
Inventory	Adds a product	A product code is generated, a row is added, the stock level decides the badge, and the chart is redrawn.
Delivery	Moves a delivery to the next stage	The badge changes, the item moves to another group, and the summary cards update.
Returns	Approves or rejects a return	The status badge changes, the available buttons change, and the refund total updates.
Reports	Changes the date range	Orders and returns are grouped again, and every figure, table and chart is rebuilt.
Users	Adds a user	A username and temporary password are generated and shown in a dialog, and the table gains a row.
Any page	The backend cannot be reached	The app quietly switches to saved data and keeps working, without telling the user.

React connection: every row above is an action plus a piece of information that changes. The search box is one piece of state and a filtered list. The current step is one piece of state. The order filters are three. The many separate screen updates that follow a single click today become one change of state and one redraw.

4. Existing JavaScript Analysis

The frontend has ten JavaScript files. The largest, `dashboard.js`, is loaded by eleven different pages and contains around 200 functions. The table below lists the browser features the code relies on most, what each one does, and which part of the screen it controls.

JavaScript	What does it do?	Page / Feature	What UI does it affect?
<code>getElementById()</code>	Finds one element on the page so its value can be read or changed	Used on every page, most heavily in the admin console and the billing counter	Cart totals, summary card values, filter dropdowns, form fields
<code>querySelector()</code>	Does the same thing, but finds the element by its style class	Header and breadcrumb, form groups	User name, role and profile letter in the header
<code>querySelectorAll()</code>	Finds a whole group of elements and loops through them	Cart, category buttons, checkout steps, form checks	Quantity buttons, the active category, the visible step, field errors
<code>addEventListener()</code>	Connects a user action to a function. Many of these sit inside drawing functions, so they are connected again every time the screen is redrawn	Billing counter, table filters, all forms	Every button, input and dropdown in the app
<code>innerHTML</code>	Builds the markup as a long piece of text and puts it on the page. This is how almost the entire admin console is drawn	<code>dashboard.js</code> , <code>ui.js</code> , <code>landing.js</code>	Whole pages, table rows, product list, cart list, dropdowns
<code>textContent</code>	Writes a single piece of text into one place on the page	Billing counter totals and labels, header, error messages	Subtotal, discount, total, hint lines, button labels
<code>classList</code>	Adds and removes style classes, and is used to remember what the app is currently doing	Checkout steps, category buttons, top bar, form checks, sidebar	Which step is showing, which category is active, whether a field looks wrong
<code>onclick</code> written inside the markup	Button actions are written into the generated text, which forces 36 functions to be made globally available	Every table row created by JavaScript	Edit, Delete, Approve and Assign buttons
<code>localStorage</code>	Stores the signed in user, the active business and a copy of the orders, stock, deliveries and returns	<code>dashboard.js</code> , <code>data.js</code> , <code>app.js</code> , <code>script.js</code>	Everything still visible after refreshing, and the backup used when the backend is offline
<code>fetch()</code>	Sends requests to the backend. <code>api-client.js</code> wraps every endpoint neatly, but <code>app.js</code> ignores it and writes its own requests eight times	Login, order checkout, loading the admin data	Whether the screen shows live or saved data

Representative example

The clearest example is the function that draws the Orders page inside `scripts/dashboard.js`. It works in five steps. First it builds the whole page, including the three filter dropdowns and an empty table, as one long piece of text and puts it on the screen. Then it searches the page to find the very elements it has just created, using their names. Next it defines a smaller function that reads the chosen values back out of the dropdowns, filters and sorts the orders, and builds the table rows as text again, writing the Edit and Delete buttons directly into that text. It then attaches a change handler to each of the three dropdowns. Finally it calls the drawing function once by hand, because nothing else will.

There are four problems in that one function. The markup is written as plain text, so no editor can check it for mistakes, and they only appear once the page is opened. The element names form an agreement between the text above and the code below that nothing enforces, so a small rename breaks the page silently. The chosen filter is stored in the dropdown on the page rather than in a variable, so there is no other record of it. And the Edit button needs a globally available function, because a button written inside text cannot reach the code around it.

React connection: JSX replaces the markup written as text. State replaces style classes and dropdown values as the place information is kept. Actions are passed straight to the button, so listeners no longer need reconnecting and no functions need to be global. Values worked out from other values, such as the cart total, no longer need to be updated by hand.

5. NestJS API Analysis

Frontend → REST API → NestJS → Database

The backend provides 63 endpoints across 12 areas. Each request carries the signed in user's role, and the server checks that role before allowing the action. The roles column below is taken from the backend code, so it shows what the server actually permits.

Feature	Method	API Endpoint	Purpose	Roles allowed
Login	POST	/api/auth/login	Authenticate user	anyone
Businesses	GET, POST, PUT, DELETE	/api/companies	List, register, update or remove a business	superuser
Staff users	GET, POST, PUT, DELETE	/api/users	Manage the staff accounts of a business	superuser, admin
Customers	GET	/api/customers/phone/{phone}	Find a customer by phone number at the counter	superuser, admin, cashier, customer
	GET, POST, PUT	/api/customers	List, create or update a customer	superuser, admin, cashier, customer
Products	GET	/api/products	Product list and categories for the counter	superuser, admin, cashier, inventorymanager, customer
	POST, PUT, DELETE	/api/products/{id}	Add, edit or delete a product	superuser, admin, inventorymanager
Inventory	GET	/api/inventory	Stock records and stock alerts	superuser, admin, inventorymanager, cashier
	POST	/api/inventory/adjust	Change the quantity held for a product	superuser, admin, inventorymanager
Orders	GET	/api/orders	Order list for the table and summary cards	superuser, admin, cashier, returnhandler
	POST	/api/orders	Create an order at checkout	superuser, admin, cashier, customer
	PUT	/api/orders/{id}	Update the status, total or payment	superuser, admin, cashier
	DELETE	/api/orders/{id}	Delete an order	superuser, admin
Bills and payments	GET, POST	/api/orders/bills, /api/orders/payments	Generate bills and record payments	superuser, admin, cashier

Feature	Method	API Endpoint	Purpose	Roles allowed
Deliveries	GET, POST	/api/deliveries	Delivery list, and creating a delivery at checkout	superuser, admin, deliveryops, cashier
	PUT	/api/deliveries/{id}	Move a delivery forward and assign a partner	superuser, admin, deliveryops
Returns	GET, POST	/api/returns	Return list, and raising a new return	superuser, admin, returnhandler, cashier
	PUT	/api/returns/{id}	Approve, reject or refund a return	superuser, admin, returnhandler
Reports	GET	/api/reports/sales, /api/reports/inventory, /api/reports/returns	Summary reports	superuser, admin, and the role concerned
Suppliers	GET, POST, PUT	/api/suppliers	Supplier details	superuser, admin, inventorymanager

Notes. Every area follows the same pattern of listing, creating, updating and deleting, so a single reusable piece of React code can serve most of the admin console. The three report endpoints already exist but are not being used, because the frontend works out the report figures itself. There is no endpoint for notifications, so those are also produced in the browser. **The existing backend will continue to be used and is not changed for this lab.**

6. Problems in the Existing Frontend

#	Current Issue	Where Does It Occur?	Why Is It a Problem?
1	A single very large JavaScript file runs eleven pages	scripts/dashboard.js, around 200 functions and 385 KB	Page switching, drawing, permissions, charts, dialogs and notifications all sit in one file. Opening the Users page downloads the whole thing, including code it will never use. Any edit risks breaking another page, and two people cannot work on two pages without clashing.
2	The sidebar and header are copied into eleven HTML files	Sidebar in 11 files, header in 12, giving 88 duplicated menu links	Renaming a page or adding a menu item means eleven identical edits. The copies have already fallen out of step, because three of the pages leave out a script file that the other eight include.
3	Pages are drawn by building markup as plain text	Used in more than 50 places across the admin console and the billing counter	Markup written as text cannot be checked by the editor, so mistakes only show up when the page is opened. Rebuilding a whole page just to change one cell also loses the scroll position and whatever the user was typing.
4	The state of the app is stored on the page itself	Style classes are used to remember the current checkout step, and the chosen filter is read back out of the dropdown	There is no single place to ask what the app is currently doing. Refreshing the billing counter quietly sends the cashier back to step one and empties the cart, and a chosen filter cannot be saved or shared.
5	Button actions are reconnected every time the screen is redrawn	The cart and the product list reconnect their buttons on every change	Typing a five letter search term rebuilds the whole product list five times. It is wasted effort and makes it easy for a button to end up firing twice.
6	Button actions have to be made globally available	36 functions are attached to the global scope so the generated markup can reach them	Because the actions are written inside text, they cannot reach the surrounding code and must be global instead. None of them can be renamed safely, and the order number is passed as text, which breaks if the value contains a quote mark.

#	Current Issue	Where Does It Occur?	Why Is It a Problem?
7	There are two different ways of talking to the backend	<code>api-client.js</code> against eight hand written requests inside <code>app.js</code>	The checkout, which is the most important flow in the product, ignores the shared helper and writes its own requests, so it repeats the server address and gets none of the error handling. Changing the address means editing two files.
8	The app falls back to old data without saying so	The admin data loader and the billing counter data layer	Any failed request is quietly replaced with saved or sample data and nothing is shown to the user. A manager can read yesterday's revenue on a dashboard that looks perfectly normal while the backend is actually down.
9	The same function is written twice	<code>renderNotificationDropdown()</code> appears at two different points in <code>dashboard.js</code>	Only the second version is ever used and the first is dead code, with no warning anywhere. Anyone editing the first one would find their changes have no effect at all.
10	The rupee symbol is broken on screen	The Revenue card on <code>dashboard.html</code> , and several places on <code>delivery.html</code>	The Revenue card currently shows " ?58,680 " instead of "₹58,680", because the file was saved with the wrong character encoding. The amount format is also inconsistent, since the billing counter shows the rupee symbol while the admin console writes "Rs" instead.

What ties these together. The first six problems are not six separate faults. They are all symptoms of the same thing, which is that there is no proper structure for reusable pieces and no proper place to keep the state of the app. Markup has nowhere to live, so it lives inside text. State has nowhere to live, so it lives in style classes. Actions have nowhere to live, so they become global. Problems 9 and 10 are the kind of mistake that grows quietly inside one very large file with no checking tools in place.

7. React Conversion Opportunities

Current approach	React idea that replaces it	Expected improvement
Sidebar and header copied into eleven files	Components, with one shared layout wrapped around every admin page	Around a thousand lines of duplicated markup become a single layout. A menu change is one edit, and the copies can no longer fall out of step.
The same card, badge, table and dialog markup written again and again	Properties passed into a component	Sixteen summary cards, forty badges and six versions of the dialog become four small components.
Pages drawn by building markup as plain text	JSX	More than fifty places that build markup as text are removed, and the editor can check the markup again.
State kept in style classes and dropdown values	State inside the component	One clear answer to what the app is currently doing, instead of reading it back off the page.
Change the data, then remember to call every update function	The screen is described from the state and redrawn automatically	Manual update calls stop existing, so the cart total can no longer disagree with the cart.
Actions reconnected on every redraw, and functions made global	The action is passed straight to the button	All of the actions written inside markup and all 36 global functions are removed.
Lists cleared and rebuilt in full	Lists drawn from the data, each item identified	Changing one quantity updates one row instead of the whole cart.
Requests written separately at each place they are needed	One reusable piece of code for loading each kind of data	Requests are handled in one place, so the eight hand written ones go through the same route as everything else.
Failures quietly fall back to old data	Loading and error states kept openly	The user can tell live figures from saved ones. The fallback stays, but it says so.
The signed in user read from browser storage all over the code	One shared source for the current user	Buttons a role is not allowed to use are never drawn at all, instead of being drawn and then switched off.

Current approach	React idea that replaces it	Expected improvement
A full page reload between eleven nearly identical documents	React Router	The hand built page switching code is removed, and the layout stays in place while moving between pages.
One very large script downloaded by every page	Separate files, loaded only when needed	Each page loads only what it actually uses, and two people can work on two pages at the same time.

8. First React Conversion Target

Selected: the Orders and Billing page. This is `front-end/pages/orders.html`, currently drawn by the `renderOrders()` function inside `scripts/dashboard.js`.

- **Small enough to finish.** The whole feature is one table, three controls and one dialog, so it can be rebuilt and compared against the original within a single lab.
- **It covers every important React idea.** Components, properties passed into them, state, user actions, lists, and loading data from the backend. Nothing important is left out, and nothing unusual is needed.
- **The state is simple.** Three filter choices and one list. The filtered and sorted rows are worked out from those, rather than stored separately.
- **The backend calls are straightforward.** Listing, creating, updating and deleting orders, with none of the chained steps that the checkout has.
- **It produces reusable pieces straight away.** The table, badge, toolbar, dialog, page heading and summary card all come out of this one page and are then reused by Inventory, Delivery, Returns and Users.
- **It fixes several of the problems listed above at once,** including the actions written inside markup, the global functions, the filter stored on the page, and the markup built as text.

Why not the others. The billing counter is the most valuable page, but it has four steps and five pieces of information that all affect each other, spread across three files. Converting it first would mean learning React and untangling a complicated flow at the same time. The dashboard would bring the charting library into React on day one. Inventory is a close second, but it also carries supplier summaries and stock adjustment. The landing page is the easiest of all, which is exactly why it would teach almost nothing.

Scope. Included: the orders table, the three filter controls, the badges, the message shown when nothing matches, the four summary cards, the New Order dialog, editing, deleting where the role allows it, and new loading and error states. Not included: the other ten admin pages, the billing counter, charts, notifications, page switching, and any change to the backend. Both versions will run side by side against the same backend during the changeover.

9. Initial React Conversion Sketch

This is a sketch for planning only. Nothing here has been built, and no page has been rebuilt.

Component	Properties passed in	Responsibility
<code><OrdersPage /></code>	none	Holds the state, loads the orders, and works out which rows to show.
<code><PageHeader /></code>	title, actions	The page title and the New Order button.
<code><FilterToolbar /></code>	value, onChange	The three controls. It holds no information of its own and reports changes upwards.
<code><DataTable /></code>	columns, rows, emptyMessage	A general purpose table and its empty message, reused later by four other pages.
<code><OrderRow /></code>	order, canDelete, onEdit, onDelete	One row. The actions are passed in, so no global function is needed.
<code><Badge /></code>	status	Decides the colour from the status, in one place.

Component	Properties passed in	Responsibility
<code><OrderFormModal /></code>	open, order, onSubmit, onClose	Creating and editing in one component. An empty order means a new one.
<code>useOrders()</code>	none	A reusable piece of code that handles the four backend calls.

How the pieces fit together

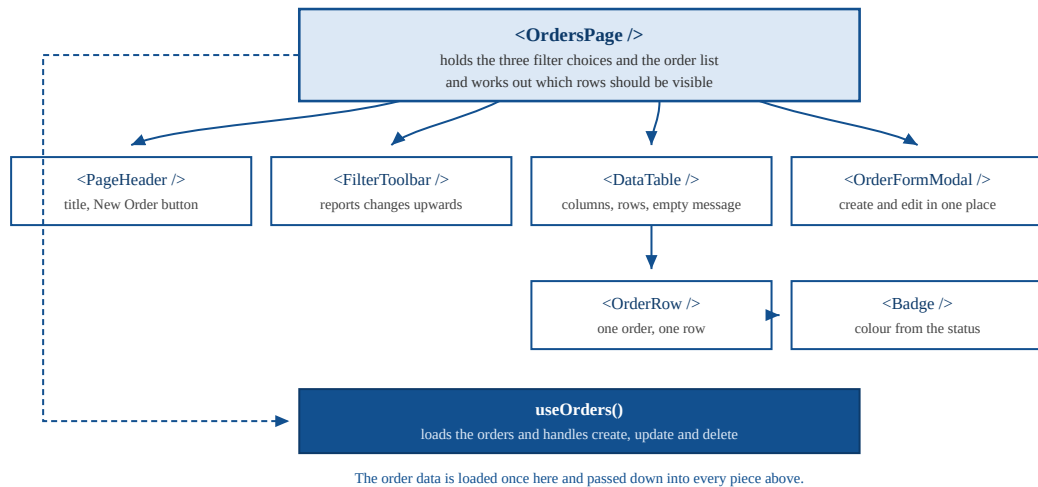


Figure 3 : The proposed structure for the Orders page. Each box is one reusable piece.

Where each piece of information lives

Information	Kept in	Replaces what is done today
The list of orders	Inside <code>useOrders()</code>	A shared list variable sitting at the top of the large JavaScript file.
Loading and error	Inside <code>useOrders()</code>	Nothing at all today, because failures are hidden from the user.
The three filter choices	State inside <code><OrdersPage /></code>	Reading the values back out of the dropdowns on the page.
Whether the dialog is open	State inside <code><OrdersPage /></code>	Separate opening and closing functions that add and remove style classes.
The rows actually shown	Worked out while drawing, never stored	A drawing function that has to be called by hand after every change.
The signed in role	One shared source for the whole app	Reading the stored user from browser storage in many separate places.

Concern	Today	After the conversion
Where the chosen filter is kept	In the dropdown on the page	In three pieces of state
How the table refreshes	Three handlers that call the drawing function by hand	Automatically, because the visible rows are worked out from the state
How Edit knows which order it is	The order number is written into the markup as text	The action is passed to that row directly
Delete for roles that may not use it	Drawn first, then switched off	Never drawn at all
A failed request	Quietly falls back to saved rows	An error message with a retry button
Reuse on other pages	Copy the drawing function and edit it	The table, badge and dialog are simply imported